

Logging in PySpark — Guide

pyspark-lakehouse-aws · background reference · 2026-08-06

Why not print()?

`print()` has no severity levels, no timestamps or source-module context, no way to turn noise up or down without editing code, and no easy way to redirect output to a file or a log-aggregation system later. Python's built-in `logging` module solves all of this and is the standard choice — no third-party library needed at this project's scale.

The four building blocks

Piece	Role
Logger	The object you call — <code>logger.info("message")</code> . Obtained via <code>logging.getLogger(__name__)</code> .
Level	Severity of a message. Anything below the configured threshold is silently dropped.
Handler	Where output goes — console (<code>StreamHandler</code>), a file (<code>FileHandler</code>), etc.
Formatter	What each output line looks like — typically timestamp, logger name, level, message.

Log levels, in order

Level	Use for
DEBUG	Verbose diagnostic detail, only useful when actively troubleshooting.
INFO	Normal pipeline milestones — "this happened, as expected."
WARNING	Unexpected but recoverable — the job continues.
ERROR	An operation failed.
CRITICAL	The application itself can't continue.

The standard idiom

```
import logging

logger = logging.getLogger(__name__)

logger.info("Starting ingestion for %s", file_name)
logger.warning("No rows found for %s", file_name)
logger.error("Failed to write %s to bronze", file_name)
```

`__name__` means the logger is named after the module it's defined in (e.g. `src.bronze.ingest_dimensions`), so every log line shows exactly where it came from with no extra work. Note the `%s` placeholders rather than an f-string — `logging` only formats the string if the message will actually be emitted at the current level, which is a small but real efficiency win over `logger.info(f"...")`, which always formats regardless of level.

Thumb rules

1 Configure logging once, at the entry point only

Every other module just calls `getLogger(__name__)` and logs — it never configures anything. Configuration happens exactly once, in `if __name__ == "__main__":`:

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s - %(message)s",
)
```

This is the single most important rule in Python logging. If every module called `basicConfig`, whichever one ran first would silently win and the rest would be no-ops. For this repo: `config/spark_session.py` and `config/schemas.py` should never touch logging configuration — only the application entry point (`ingest_dimensions.py`'s `main()`) should.

2 Log at pipeline milestones, not inside loops

Log at boundaries: "starting ingestion for X," "read N rows," "wrote to bronze," "failed: reason."
Never log per-row — in a distributed job that's not just noisy, it can actively hurt performance.

3 Driver logs vs. executor logs are not the same thing

A `logger.info(...)` call inside driver-side code (your `ingest_dimensions` function itself) shows up in your terminal as expected. But if that call ends up inside a **UDF** — code that runs on executors, not the driver — it won't appear where you expect. Executor output goes to separate executor log files, not the driver's console. Keep logging in driver-level code, before/after DataFrame actions, not inside per-row transformation logic.

4 There are two separate logging systems in a PySpark app

Everything above is your **application's** logging (Python's `logging` module). Spark itself also logs its own internal engine chatter — task scheduling, shuffle stats, executor status — via **log4j2**, on the JVM side. That's the wall of `INFO` text that floods the terminal on every `spark-submit` run. It's configured separately, in `$SPARK_HOME/conf/log4j2.properties` (copy from the `.template` file), typically setting `rootLogger.level = warn` so it doesn't drown out your own log lines.

5 Use `.exception()`, not `.error()`, inside an except block

```
try:
    dimensions_df.write.mode("overwrite").csv(f"s3a://bronze/{file_name}/")
except Exception:
    logger.exception("Failed to write %s to bronze", file_name)
    raise
```

`logger.exception(...)` automatically attaches the full stack trace to the log output.
`logger.error(...)` does not — a small but genuinely useful API detail, valid only when called from inside an `except` block.

6 Never log secrets

Directly relevant to this repo: `config/spark_session.py` reads `MINIO_ROOT_USER / MINIO_ROOT_PASSWORD` from the environment. A debug line that dumps the full Spark conf, or logs a connection string, would leak credentials into your terminal or log files.

LEAKS A SECRET

```
logger.debug("Spark conf: %s",
spark.sparkContext.getConf().getAll(
))
```

STATES INTENT ONLY

```
logger.info("Configured S3A endpoint
for MinIO")
```

Full example — how this would look wired into this repo

Illustrative shape, not a drop-in — for you to write and adapt yourself.

config/logging_config.py

```
import logging

def configure_logging(level: int = logging.INFO) -> None:
    logging.basicConfig(
        level=level,
        format="%(asctime)s %(levelname)s %(name)s - %(message)s",
    )
```

src/bronze/ingest_dimensions.py

```
import logging

from config.logging_config import configure_logging
from config.schemas import SCHEMAS
from config.spark_session import get_spark_session

logger = logging.getLogger(__name__)

def ingest_dimensions(file_name, schema):
    spark = get_spark_session()

    logger.info("Reading %s", file_name)
    dimensions_df = (
        spark.read.schema(schema)
        .option("multiline", "true")
        .csv(f"data/raw/{file_name}.csv", header=True)
    )

    row_count = dimensions_df.count()
    logger.info("Read %d rows for %s", row_count, file_name)

    try:
        dimensions_df.write.mode("overwrite").csv(f"s3a://bronze/{file_name}/")
    except Exception:
        logger.exception("Failed to write %s to bronze", file_name)
        raise

    logger.info("Wrote %s to bronze", file_name)

def main():
    configure_logging()
    for file_name, schema in SCHEMAS.items():
        ingest_dimensions(file_name, schema)

if __name__ == "__main__":
    main()
```

Note `configure_logging()` is only called once, inside `main()` — consistent with Thumb Rule 1.

Official documentation

- **Python logging HOWTO** (best starting point) — docs.python.org/3/howto/logging.html
- **Python logging module reference** — docs.python.org/3/library/logging.html
- **Python logging cookbook** (advanced recipes: multiple handlers, JSON formatting, etc.) — docs.python.org/3/howto/logging-cookbook.html
- **Apache Spark — configuring logging (log4j2)** — spark.apache.org/docs/latest/configuration.html#configuring-logging